

13주차

≡ 다중 선택

개념

7. Transformer

1. BERT
2. BART
3. ELECTRA
4. T5
5. 모델 종합 비교

1. BERT

BERT(Bidirectional Encoder Representations from Transformers) → 2018년 구글이 발표한 언어 모델. 트랜스포머 기반 **양방향 인코더**를 사용하는 자연어 처리 모델이다.

1-1. 양방향(Bidirectional) 인코더란?

기존의 언어 모델은 문장을 **왼쪽에서 오른쪽으로 순차적으로** 학습한다. 따라서 단어의 의미와 문맥을 파악할 때 **이전 단어만** 고려할 수 있었다.

반면 BERT는 문장을 **양방향(좌우 동시)**으로 학습하므로, **이전 단어와 이후 단어를 모두 참조**해 단어의 의미와 문맥을 파악할 수 있다.

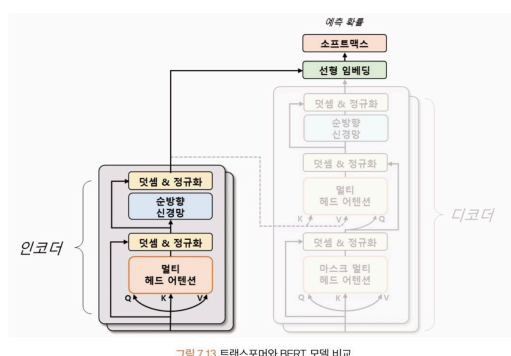
예시

문장: "나는 강에서 낚시를 했다"

단방향 모델: "낚시" 예측 시 → "나는", "강에서" 만 참조 가능

BERT : "낚시" 예측 시 → "나는", "강에서", "를", "했다" 모두 참조 가능

이를 통해 BERT는 기존 모델보다 더 정확하게 문맥을 파악하고, 다양한 자연어 처리 작업에서 높은 성능을 보인다.



BERT는 트랜스포머의 **인코더 모듈만** 사용한다. 트랜스포머는 인코더-디코더 구조를 모두 갖추고 있지만, BERT는 문장의 의미와 구조를 이해하는 것이 목적이므로 디코더는 사용하지 않는다.

BERT는 대규모 데이터를 사용해 **사전 학습**되어 있으므로 **전이 학습(Transfer Learning)**에 주로 활용된다. 일부 또는 전체를 다른 작업에서 재사용하면 적은 양의 데이터로도 높은 정확도를 달성할 수 있으며, 모델 학습 시간을 크게 단축할 수 있다.

1-2. 사전 학습 방법

BERT는 사전 학습을 위해 두 가지 방법을 사용한다: **마스킹된 언어 모델링(MLM)**과 **다음 문장 예측(NSP)**

① 마스킹된 언어 모델링 (MLM, Masked Language Modeling)

입력 문장에서 임의로 일부 단어를 마스킹하고, 해당 단어를 예측하는 방식이다.

```
원본 : "I'm learning PyTorch"
마스킹: "I'm [MASK] PyTorch"
BERT가 [MASK]에 들어갈 단어 'learning'을 예측
```

이 과정에서 양방향 문맥 정보를 참고한다. 이를 통해 BERT는 문장에서 누락된 단어를 추론하는 능력을 갖게 되고, 문장 전체 의미를 이해하는 능력이 향상된다.

마스킹 토큰 적용 방법 — 전체 토큰의 15%를 선택한 후:

처리 방식	비율	이유
[MASK] 토큰으로 대체	80%	실제 마스킹 학습
어휘 사전의 무작위 단어로 변경	10%	모델이 문맥을 더 적극적으로 활용하도록 유도
실제 토큰 그대로 유지	10%	모델이 무조건 예측을 바꾸지 않도록 방지

단순히 항상 [MASK] 토큰으로만 대체하면, 실제 파인튜닝 시 [MASK] 토큰이 없으므로 사전 학습과 미세 조정 간의 불일치가 발생한다. 10%를 무작위 단어로, 10%를 원래 단어로 유지함으로써 이 불일치를 줄인다.

② 다음 문장 예측 (NSP, Next Sentence Prediction)

두 개의 문장이 주어졌을 때, 두 번째 문장이 첫 번째 문장의 다음에 오는 문장인지 여부를 판단하는 작업이다.

예시

```
문장 A: "I'm learning PyTorch"
문장 B: "PyTorch is a machine learning library" → IsNext (연속적)

문장 A: "I'm learning PyTorch"
문장 B: "The weather is nice today" → NotNext (무관)
```

이 과정에서 BERT는 두 문장 간의 관계를 학습하고, 문장 간의 의미적인 유사성을 파악한다. 주로 질의응답(QA), 자연어 추론(NLI) 등 두 문장 간 관계 이해가 필요한 작업에 효과적이다.

1-3. 특수 토큰

BERT 모델은 입력 문장에 특수한 토큰들을 추가해 모델이 학습하고 추론하는 과정에서 필요한 정보를 제공한다.

토큰	삽입 위치	역할
[CLS]	문장 맨 앞	문장 분류 작업에 사용하는 토큰. 이 토큰의 벡터가 문장 전체의 의미를 담게 되어

토큰	삽입 위치	역할
		분류기 입력으로 사용됨
[SEP]	문장과 문장 사이, 문장 끝	두 개 이상의 문장을 구분하기 위해 사용. 두 문장 분류 시 각 문장 뒤에 삽입
[MASK]	마스킹된 단어 위치	입력 문장 내에서 임의로 선택된 단어를 가리키는 특별한 토큰. MLM 학습에 사용

따라서 BERT 모델의 입력 시퀀스는 다음과 같은 구조를 갖는다:

[CLS] 문장-1 [SEP] 문장-2 [MASK] [SEP]

1-4. 입력 임베딩 구성 (3가지 합산)

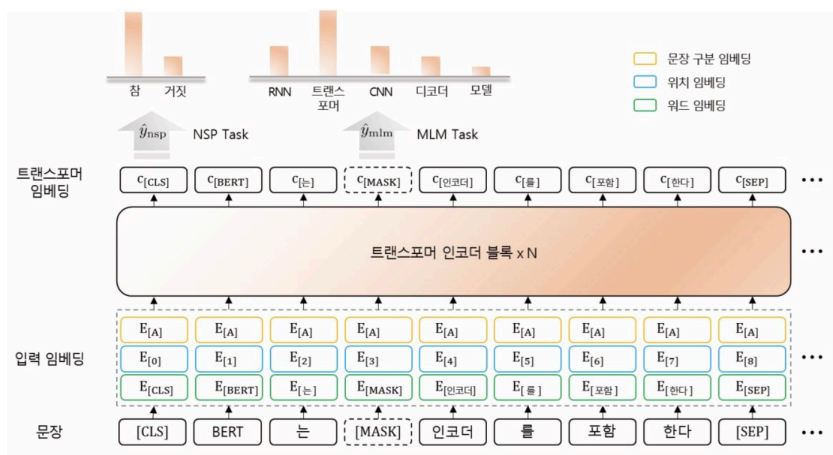


그림 7.14 BERT 모델의 입력 임베딩과 손실 함수

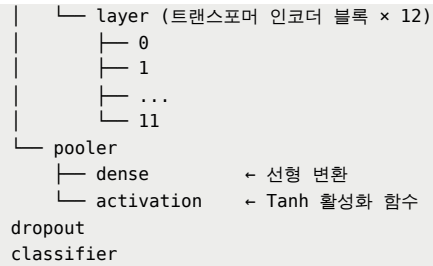
BERT의 입력 임베딩은 세 가지 임베딩을 **합산(element-wise sum)** 하여 구성한다.

임베딩 종류	역할	비고
토큰 임베딩 (word_embeddings)	각 단어 자체의 의미 벡터	WordPiece 토큰나이저 사용
위치 임베딩(position_embeddings)	문장 내 단어의 순서(위치) 정보	트랜스포머의 positional encoding과 동일한 역할
문장 구분 임베딩 (token_type_embeddings)	해당 토큰이 첫 번째 문장인지 두 번째 문장인지 구분	NSP 학습에 필요한 정보

이렇게 세 임베딩을 합산한 후, **레이어 정규화(LayerNorm)** 와 **드롭아웃(Dropout)** 을 통과시켜 최종 입력 임베딩 벡터를 생성한다.

1-5. 모델 내부 구조

```
bert
├── embeddings
│   ├── word_embeddings      ← 토큰 임베딩
│   ├── position_embeddings  ← 위치 임베딩
│   └── token_type_embeddings ← 문장 구분 임베딩
├── LayerNorm
├── dropout
└── encoder
```



각 구성 요소 설명

- **인코더 블록 × 12개**: 각 블록은 셀프 어텐션(Self-Attention)과 순방향 신경망(Feed-Forward Network)으로 구성되어, 입력 시퀀스를 순차적으로 처리하며 새로운 임베딩을 생성한다. 각 블록은 셀프 어텐션 및 순방향 신경망을 사용해 입력 시퀀스의 다양한 관계를 학습한다.
- **풀러(Pooler)**: [CLS] 토큰 벡터를 한 번 더 비선형 변환하기 위해 **선형 변환(dense)** 과 **Tanh 활성화 함수**를 사용한다. 이후 드롭아웃이 적용되어 모델의 과대적합을 방지한다.
- **분류기(Classifier)**: BERT 모델에서 수행해야 하는 작업으로, [CLS] 토큰 벡터를 활용해 결과를 예측한다. 분류 문제에서는 일반적으로 [CLS] 토큰 벡터를 사용하고, 각 토큰마다 예측이 필요한 경우에는 모든 토큰 벡터를 사용한다.

BertTokenizer 주요 설정

```
tokenizer = BertTokenizer.from_pretrained(
    "bert-base-multilingual-cased",
    do_lower_case=False # 소문자 변환 안 함 → 대소문자 구분 유지
)
```

- **bert-base-multilingual-cased**: 104개 언어를 지원하는 다국어 사전 학습 BERT 모델
- **do_lower_case=False**: 'Apple'과 'apple'을 다른 단어로 인식

최적화 함수: AdamW

```
optimizer = optim.AdamW(model.parameters(), lr=1e-5, eps=1e-8)
```

- **AdamW**: Adam 최적화 함수에 가중치 감쇠를 추가한 변형된 경사 하강법 알고리즘
- **eps**: 학습률을 0으로 나누는 것을 방지하기 위해 분모에 더하는 작은 값
- BERT는 대규모의 매개변수를 가진 딥러닝 모델이므로 안정적인 기울기 갱신이 필수적. AdamW는 갱신되는 모든 매개변수의 학습률을 조절하는 적응형 활성화 함수이므로 모델이 빠르게 수렴하고 안정적으로 학습할 수 있다.

1-6. 미세 조정 (Fine-tuning)

사전 학습을 수행한 후, 미세 조정 기법을 통해 다양한 자연어 처리 작업에 적용할 수 있다.

적용 가능한 작업	설명
문장 분류	입력 문장이 긍정/부정인지, 또는 특정 카테고리인지 분류
감성 분석	텍스트에서 긍정/부정/중립 감성 파악
질문 응답	주어진 지문에서 질문에 대한 답변 추출

적용 가능한 작업	설명
기계 번역	한 언어에서 다른 언어로 번역
개체명 인식(NER)	텍스트에서 인명/지명/기관명 등 개체 추출

미세 조정 과정에서는 해당 작업에 맞는 계층을 추가하고 손실 함수를 정의해 학습한다.

2. BART

BART(Bidirectional Auto-Regressive Transformer) → 2019년 메타(Meta)의 FAIR 연구소에서 발표한 트랜스포머 기반 모델

2-1. 핵심 아이디어

BART는 **BERT**의 인코더와 **GPT**의 디코더를 결합한 **시퀀스-시퀀스(Seq2Seq)** 구조로, **노이즈 제거 오토인코더(Denoising Autoencoder)**로 사전 학습된다.

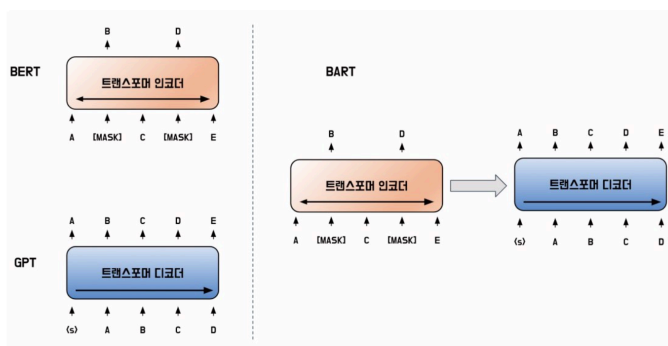


그림 7.15 BART 사전 학습 시각화

BERT / GPT / BART 구조 비교

모델	구조	문맥 방향	주요 강점
BERT	인코더만	양방향	문맥 이해, 분류
GPT	디코더만	단방향(순방향)	텍스트 생성
BART	인코더 + 디코더	양방향 인코더 + 순방향 디코더	이해 + 생성 모두

인코더를 사용함으로써 순방향 정보만 인식할 수 있는 GPT의 단점을 개선해 양방향 문맥 정보를 반영할 수 있으며, 디코더를 사용함으로써 문장 생성 분야에서 뛰어나지 않았던 BERT의 단점을 개선했다.

2-2. 사전 학습: 노이즈 제거 오토인코더

오토인코더 사전 학습 방법은 입력 데이터에 잡음을 추가하고, 잡음이 없는 원본 데이터를 복원하도록 학습하는 방식으로 수행된다.

- **노이즈가 추가된 텍스트** → 인코더에 입력
- **원본 텍스트** → 디코더에 입력
- 디코더가 원본 텍스트를 생성할 수 있게 학습

이는 BERT가 인코더 학습 시 텍스트에 마스킹을 추가해 노이즈를 주는 방법과, GPT가 디코더 학습 시 다음에 등장할 단어를 예측하는 방법으로 간주할 수 있다.



BART의 인코더는 이러한 다양한 노이즈 기법을 통해 입력 문장의 변형에 강건해지고, 디코더는 원본 문장을 복원하는 과정에서 자연스러운 텍스트 생성 능력을 갖추게 된다.

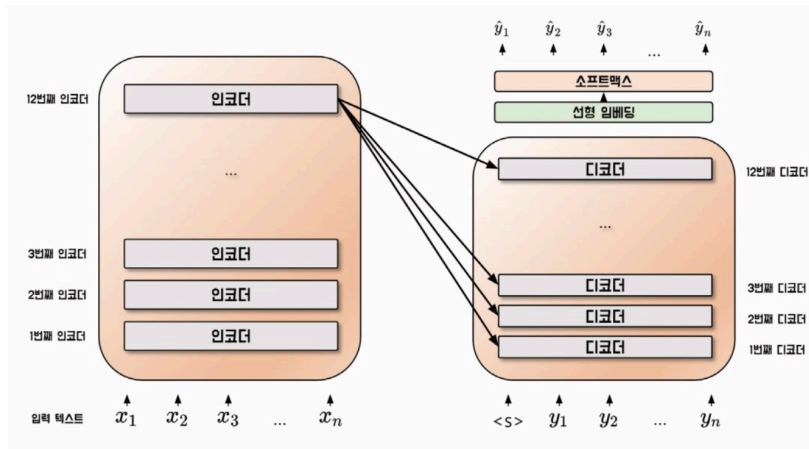


그림 7.16 BART 모델 구조

BART는 인코더와 디코더를 모두 사용하므로 트랜스포머와 유사한 구조를 가지고 있지만, 차이점이 존재한다.

트랜스포머 vs BART 어텐션 방식 비교

	트랜스포머	BART
어텐션 연산 위치	인코더 모든 계층 ↔ 디코더 모든 계층	인코더 마지막 계층 ↔ 디코더 각 계층
장점	세밀한 정보 전달	정보 전달 최적화, 메모리 사용량 절감

BART의 인코더에서는 입력 문장의 각 단어를 임베딩하고 여러 층의 인코더를 거쳐 마지막 계층에서는 입력 문장 전체의 의미를 가장 잘 반영하는 벡터가 생성된다. 이렇게 생성된 벡터는 디코더가 출력 문장을 생성할 때 참고되며, 디코더의 각 계층에서는 이 벡터와 이전 계층에서 생성된 출력 문장의 정보를 활용해 출력 문장을 생성한다.

BART 모델 내부 구조 (bart-base 기준)

```

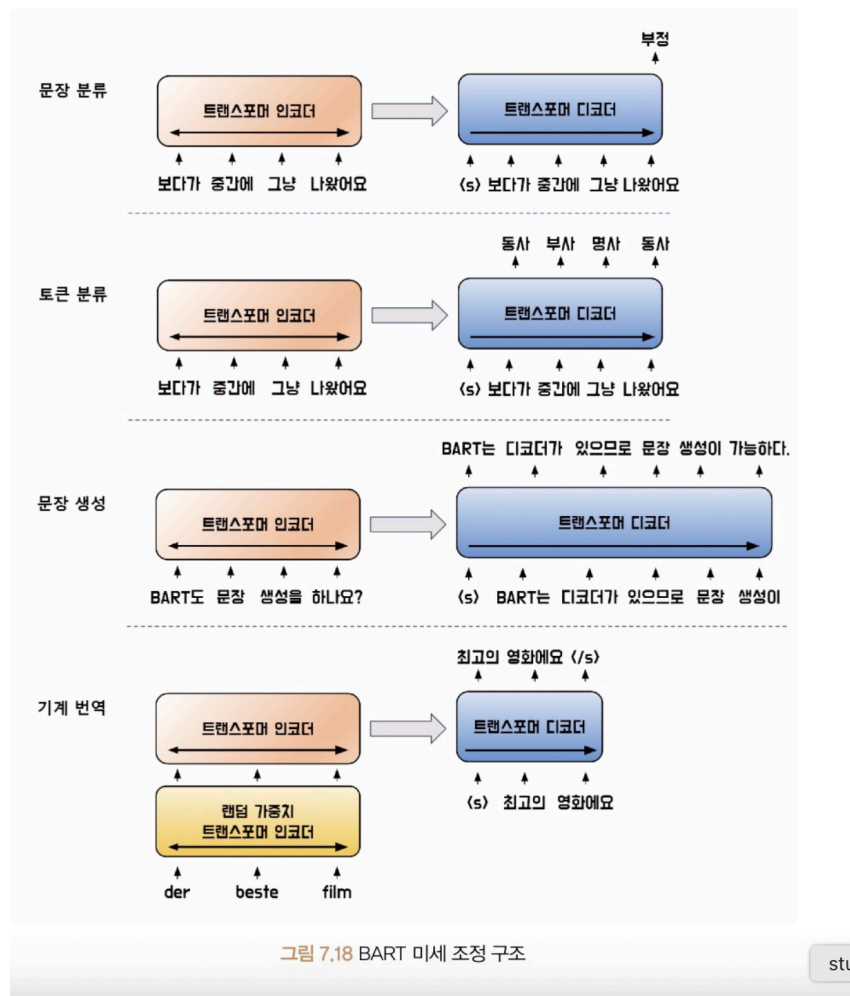
model
├── shared          ← 인코더·디코더가 공유하는 임베딩 계층
├── encoder
│   ├── embed_tokens
│   ├── embed_positions
│   └── layers (× 6)
│       └── layernorm_embedding
├── decoder
│   ├── embed_tokens
│   ├── embed_positions
│   └── layers (× 6)
│       └── layernorm_embedding
└── lm_head
  
```

- **shared 계층:** 인코더와 디코더가 공유하는 임베딩 계층을 의미하며, 이러한 공유로 인코더와 디코더 간의 연결을 강화한다.
- **layernorm_embedding:** 인코더와 디코더에서 각 토큰의 임베딩에 적용되는 계층 정규화로 임베딩 벡터의 마지막 차원에 대해 정규화를 수행해 학습을 안정화한다.
- **lm_head:** 마지막 디코더 계층의 출력값은 출력 크기가 단어 사전의 크기인 완전 연결 계층을 통과해 언어 모델을 형성한다.

bart-base 는 6개의 인코더/디코더 계층을 사용하며, 12개 계층을 사용하는 모델은 **bart-large** 로 불러올 수 있다.

2-5. 미세 조정 구조 (작업별)

BART는 인코더와 디코더를 모두 사용하는 구조를 가지고 있기 때문에 미세 조정 시 각 다운스트림 작업에 맞게 입력 문장을 구성해야 한다. 즉, 인코더와 디코더에 다른 문장 구조로 입력한다.



작업	입력 방식	출력 방식
문장 분류	인코더·디코더에 동일한 문장 입력	디코더의 마지막 토큰 은닉 상태 → 선형 분류기. BERT의 [CLS]와 비슷하지만, BART는 전체 입력과 어텐션 연산이 적용된 은닉 상태 사용
토큰 분류	인코더·디코더에 동일한 문장 입력	디코더의 각 시점별 마지막 은닉 상태 → 토큰 분류기. 전체 입력과 디코더 각 시점별 은닉 상태와의 어텐션 연산 수행
문장 생성	인코더에 입력 문장 입력	디코더가 출력 문장을 생성
기계 번역	사전 학습된 인코더 + 새로운 번역용 인코더 추가. 새 인코더는 기존 단어 사전 없이도 되며, 디코더는 사전 학습된 가중치와 단어 사전 사용	두 단계 학습: ① 새 인코더 가중치·위치 임베딩·첫 번째 인코더 층의 셀프 어텐션 입력 행렬 가중치만 학습 → ② 모든 신경망의 가중치를 작은 반복으로 학습

BART는 트랜스포머 디코더를 사용하기 때문에 BERT가 해결하지 못했던 문장 생성 작업을 수행할 수 있다. 특히 입력값을 조작해 출력을 생성하는 **추상적 질의응답(Abstractive QA)** 과 **문장 요약(Summarization)** 같은 작업에 적합하다.

패딩값 설정: -100

BART 학습 시 레이블의 패딩값으로 **-100** 을 사용한다. 이는 교차 엔트로피와 같은 손실 함수에서 패딩된 토큰을 무시하게 하기 위해 사용한다. 패딩된 토큰은 모델의 출력값과 비교하지 않고 실제 레이블을 가진 토큰만 손실을 계산하는 데 사용한다.

2-6. 평가 지표: ROUGE 점수

문장 요약 작업에서는 정확도 대신 **ROUGE (Recall-Oriented Understudy for Gisting Evaluation)** 점수를 사용한다.

ROUGE 점수는 생성된 요약문과 정답 요약문이 얼마나 유사한지를 평가하기 위해 **토큰의 N-gram 정밀도와 재현율**을 이용해 평가하는 지표다.

ROUGE 종류

지표	기준	설명
ROUGE-N	N-gram	N=1이면 유니그램(ROUGE-1), N=2면 바이그램(ROUGE-2)
ROUGE-L	LCS (최장 공통부분 수열)	가장 길게 연속되는 N-gram을 식별. 요약 문장이 입력 문장의 의미를 잘 전달하는지 평가에 유용
ROUGE-LSUM	union-LCS	ROUGE-L의 변형. 각 문장 쌍의 LCS를 합집합 한 것으로 중복 부분을 제거한 후 길이를 계산. 요약의 정확성과 완전성을 모두 반영
ROUGE-W	가중치 적용 LCS	연속된 LCS에 가중치를 부여해 계산. 단어 간 유사도까지 고려해 요약 문장이 입력 문장의 의미를 더욱 잘 전달하는지 평가

ROUGE-2는 0에서 1 사이의 값을 가지며, 1에 가까울수록 높은 성능을 의미한다.

3. ELECTRA

ELECTRA(Efficiently Learning an Encoder that Classifies Token Replacements Accurately) → 2020년 구글에서 발표한 트랜스포머 기반 모델.

3-1. 기존 MLM 방식의 한계

BERT를 비롯한 많은 자연어 처리 모델은 마스킹된 언어 모델링(MLM) 방식을 사용하거나 일부 토큰을 [MASK]로 대체하여 입력을 손상시키고 원래 토큰을 복원하는 모델을 학습한다.

MLM의 문제점: 입력 토큰 전체 중 **15%만** 마스킹되어 학습에 활용된다. 즉, 한 번의 forward pass에서 전체 토큰의 85%는 학습에 직접적으로 기여하지 않는다. → **학습 효율이 낮음**

ELECTRA는 이 문제를 해결하기 위해 **생성자(Generator)**와 **판별자(Discriminator)**를 사용해 사전 학습을 수행한다.

3-2. RTD: 생성자 + 판별자

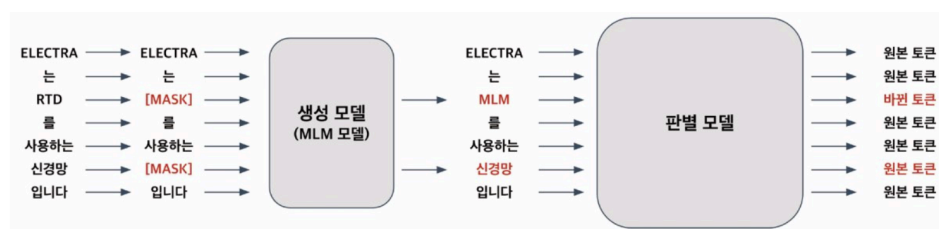


그림 7.20 ELECTRA 모델 학습 방법

① 생성자 모델 (Generator) — MLM 모델

- BERT의 마스킹된 언어 모델과 동일한 구조
- 입력 문장의 **15%를 마스크 처리**
- 마스크 처리된 토큰이 원래 어떤 토큰이었는지를 예측하며 학습

② 판별자 모델 (Discriminator) — RTD 모델

- 생성자 모델이 복원한 결괏값을 입력받아 **각 토큰이 원본 토큰인지 생성자가 바꾼 토큰인지 판별**
- 모든 토큰에 대해 이진 분류를 수행하므로, **전체 토큰이 학습에 기여** → 높은 학습 효율
- 이 학습 방법을 **RTD(Replaced Token Detection)** 이라고 함

ELECTRA 학습의 전체 흐름

1. 원본 문장 일부 토큰을 [MASK]로 대체
2. 생성자가 [MASK] 위치의 토큰을 예측해 문장 복원
3. 판별자가 복원된 문장의 각 토큰이 원본인지 생성된 것인지 판별
4. 두 모델을 동시에 학습
5. 사전 학습 완료 후, 판별자 모델만 남겨 다운스트림 작업에 사용

3-3. GAN과의 차이점

ELECTRA의 학습 방식은 생성적 적대 신경망(GAN)과 유사하지만, 몇 가지 중요한 차이점이 있다.

비교 항목	GAN	ELECTRA
생성자가 원래 토큰 정확히 예측 시	생성된 것으로 간주	원본 토큰으로 간주
학습 방식	생성자와 판별자가 적대적 으로 학습 (판별자를 속이려는 생성자 vs 구분하려는 판별자)	생성자는 MLM으로, 판별자는 RTD로 각각 독립적 으로 학습
생성자 입력	완전한 노이즈 벡터	일부 마스크 처리된 텍스트

이 차이 덕분에 ELECTRA는 GAN의 불안정한 학습 문제 없이 효율적인 사전 학습이 가능하다.

3-4. 가중치 공유 전략

생성 모델과 판별 모델은 모두 트랜스포머 인코더 구조를 따른다. 따라서 두 모델이 같은 개수의 계층으로 구성돼 있다면 가중치를 공유할 수 있어 더 빠르게 학습할 수 있다.

문제: 두 모델이 가중치를 완전히 공유하면 생성 모델의 성능이 너무 높아져서 판별 모델이 학습할 수 없게 된다.

해결 방법:

- 생성 모델을 판별 모델 크기의 **1/2에서 1/4** 크기로 바꿔 설정
- 모든 가중치를 공유하는 대신 **임베딩 계층의 가중치만 공유**

이를 통해 판별 모델이 학습하기 적합한 생성 모델을 유지하면서도 가중치를 공유하여 효율적으로 학습할 수 있다.

3-5. 다운스트림 활용

ELECTRA는 사전 학습이 완료되면 **생성 모델을 사용하지 않고 오직 판별 모델만 사용해** 다운스트림 작업을 수행한다.

- 판별 모델은 트랜스포머 인코더로 구성되어 있으며, BERT와 동일한 구조를 갖는다
- 다운스트림 작업은 BERT와 동일한 방식으로 미세 조정한다

GLUE 벤치마크 비교

GLUE(General Language Understanding Evaluation)는 문장 수준 또는 문서 수준의 이해력을 평가하는 데이터셋으로, 문장 분류, 문장 유사도 계산, 자연어 추론, 질의응답 등 11가지 과제를 통해 모델의 성능을 평가한다.

모델 비교	ELECTRA 우위
electra-small vs bert-small	+5%
electra-base vs bert-base	+3%
electra-large vs bert-large	+2%

또한 **12시간 학습한 electra-small이 4일 학습한 bert-small보다 더 높은 GLUE 점수를 기록했다.** → ELECTRA가 더 효율적으로 학습됨

4. T5

T5(Text-to-Text Transfer Transformer) → 2019년 구글에서 발표한 자연어 처리 분야의 딥러닝 모델. 트랜스포머 구조를 기반으로 한다.

4-1. Text-to-Text 구조

기존의 자연어 처리 모델은 대부분 입력 문장을 벡터나 행렬로 변환한 뒤, 이를 이용해 출력 문장을 생성하는 방식이거나 출력값이 클래스 또는 입력값의 일부를 반환하는 형식으로 동작했다.

이와 달리 T5는 **입력과 출력을 모두 토큰 시퀀스(텍스트)로 처리하는 텍스트-텍스트(Text-to-Text) 구조다.** 따라서 입력과 출력의 형태를 자유롭게 다룰 수 있으며, 모델 구조상 유연성과 확장성이 뛰어나기 때문에 새로운 자연어 처리 작업에서도 쉽게 적용할 수 있다.

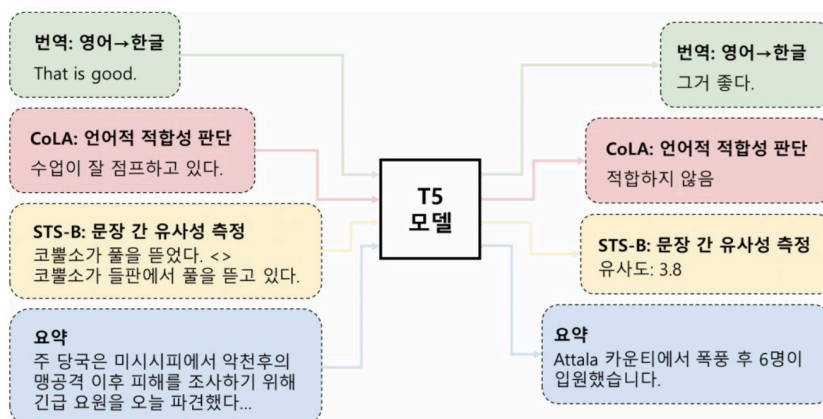


그림 7.21 T5 모델 학습 유형

T5 학습 작업 예시

작업	입력 예시	출력 예시
번역 (영→한)	translate English to Korean: That is good.	그거 좋다.
언어 수용성 판단 (CoLA)	cola sentence: 수업이 잘 점프하고 있다.	적합하지 않음
문장 유사도 측정 (STS-B)	sts sentence1: ... sentence2: ...	유사도: 3.8
문서 요약	summarize: [긴 뉴스 본문]	[짧은 요약문]

- *작업 토큰(Task Token)**의 역할:
- 문장이 인코딩되기 전에 **작업 토큰을 문장 앞에 추가**한다
- T5는 다양한 작업을 하나의 모델로 처리하므로, 작업 토큰을 통해 현재 수행할 작업이 무엇인지 모델에 알려준다
- 작업 토큰도 함께 학습하여 다양한 자연어 처리 작업에서 높은 성능을 발휘한다

4-2. 사전 학습 방식

학습 데이터: C4(Colossal Clean Crawled Corpus) — 대규모 웹 크롤링 데이터셋

사전 학습 방법: 비지도 학습 방식 (스팬 마스킹)

- 입력 문장의 일부 구간(span)을 마스킹해 입력 시퀀스를 처리
- 출력 시퀀스는 실제 마스킹된 토큰과 마스크 토큰의 연결로 구성

센티넬 토큰(Sentinel Token):

- 문장마다 유일한 마스크 토큰을 의미하는 특수 토큰
- `<extra_id_0>`, `<extra_id_1>` 형태로 0부터 99까지 **100개**의 기본값을 사용

예시

원본 문장 : "인코더-디코더 모델 구조"

마스킹 후 입력 토큰 : [`<extra_id_0>`, -, `<extra_id_1>`, 모델 구조]

출력 토큰 : [인코더, `<extra_id_0>`, 디코더, `<extra_id_1>`]

T5는 이러한 마스킹 토큰을 예측하는 것을 목적으로 사전 학습되며, 사전 학습이 완료된 후 미세 조정을 통해 다양한 자연어 처리 작업을 수행한다.

4-3. 미세 조정

T5의 미세 조정은 **지도 학습 방식**으로 학습된다.

- 번역, 언어 수용성, 문장 유사도, 문서 요약 등 다양한 작업에 활용 가능
- T5 모델의 입력과 출력은 텍스트 시퀀스 토큰들로 이루어져 있으며, 인코더-디코더 T5 모델에 입력된다
- 이 과정에서 문장이 인코딩되기 전에 `'translate English to German: '` 또는 `'summarize: '` 와 같은 작업 토큰을 문장 앞에 추가한다
- 이러한 방식은 작업 토큰도 함께 학습해 다양한 자연어 처리 작업에서 높은 성능을 발휘할 수 있게 된다

BART vs T5 미세 조정 비교

	BART	T5
입력 구성	인코더와 디코더에 다른 문장 구조	인코더에 작업토큰: 입력문장 형태

	BART	T5
출력 처리	레이블 패딩값 -100 사용	입력과 동일한 텍스트 시퀀스
디코더 입력	<code>decoder_input_ids</code> 별도 구성 필요	자동 처리

4-4. 모델 구조

T5ForConditionalGeneration 클래스는 T5 모델의 미세 조정을 위한 클래스다.

```

shared      ← 인코더·디코더 공유 임베딩 (토큰 임베딩 가중치 공유)
encoder
├── embed_tokens
├── block (× 6, t5-small 기준)
│   └── layer
│       ├── 0
│       └── ...
└── final_layer_norm / dropout
decoder
├── embed_tokens
├── block (× 6)
│   └── layer
└── final_layer_norm / dropout
lm_head      ← 최종 출력 (단어 사전 크기의 완전 연결 계층)

```

- **shared 임베딩**: 인코더와 디코더 함수에 사용되는 토큰 임베딩으로 서로 공유되게 설정된다. 인코더와 디코더의 각 계층에서 동일한 가중치가 사용되어 모델의 매개변수 수를 줄이고 일반화 성능을 향상시킨다.
- **t5-small**: T5 모델 중 가장 작은 크기. 트랜스포머와 동일한 구조.
- **t5-large**: 12개 계층 사용

학습 시 주요 매개변수

T5 모델은 토큰 인덱스(input_ids), 어텐션 마스크(attention_mask), 디코더 토큰 인덱스(decoder_input_ids), 라벨(labels)로 모델을 학습한다.

- **decoder_input_ids**: 디코더에 입력될 시퀀스를 인코딩한 텐서로, 마지막 토큰을 제외한 나머지 토큰을 사용
- **labels**: decoder_input_ids보다 다음 시점을 예측하도록 `target_ids[:, 1:]` 로 설정. pad_token_id 토큰은 -100으로 설정해 손실값 계산 시 무시되게 설정
- **contiguous()**: 텐서를 메모리상에 연속된 블록으로 저장하는 역할을 한다. 이를 통해 메모리상에 인접한 위치에 저장된 데이터를 빠르게 연산할 수 있다.

생성(generate) 메서드 주요 파라미터

파라미터	설명
<code>input_ids</code> , <code>attention_mask</code>	입력 문장의 인코딩과 마스크 정보
<code>max_length</code>	생성될 요약문의 최대 길이
<code>num_beams</code>	빔 서치(Beam Search) 알고리즘의 빔 크기. 디코더 모델이 생성한 다수의 후보 단어 시퀀스 중 가장 높은 확률을 가진 시퀀스를 선택
<code>repetition_penalty</code>	중복 토큰 생성을 제어. 값이 높을수록 중복 억제
<code>length_penalty</code>	생성된 시퀀스 길이에 대한 보상 제어. 값이 높을수록 더 긴 시퀀스 생성
<code>early_stopping</code>	최대 길이 도달 전 EOS 토큰이 생성되면 중단

5. 모델 종합 비교

구조 및 사전 학습 방법 비교

모델	발표	기관	구조	사전 학습 방법	주요 활용
BERT	2018	Google	인코더만	MLM + NSP	문맥 이해, 분류, QA
BART	2019	Meta (FAIR)	인코더 + 디코더	노이즈 제거 오토인코더 (5가지 노이즈)	요약, 번역, 생성
ELECTRA	2020	Google	인코더 (판별자)	RTD (생성자+판별자)	효율적 분류, 이해
T5	2019	Google	인코더 + 디코더	스팬 마스킹 (비지도)	모든 NLP 작업 통합

핵심 개념 흐름 정리

BERT

- 양방향 인코더 + MLM/NSP 사전학습으로 문맥 이해력 획기적 향상
 - 한계: 디코더가 없어 문장 생성 불가

BART

- BERT 인코더 + GPT 디코더 결합 → 이해 + 생성 모두 가능
 - 노이즈 제거 오토인코더로 다양한 변형에 강건하게 학습
 - 요약·번역·생성에 특화

ELECTRA

- MLM의 학습 효율 문제 해결 → GAN 아이디어 도입
 - 생성자(15% 마스킹 복원) + 판별자(전체 토큰 RTD)
 - 동일 시간 대비 더 높은 성능, 더 적은 파라미터

T5

- 모든 NLP 작업을 "텍스트→텍스트"로 통일
 - 작업 토큰(summarize:, translate:)으로 단일 모델로 범용 처리
 - 입출력 형태의 유연성·확장성 극대화